LLM Gateway — Docker & Deployment

Interview preparation · V1 · three-service Compose stack, migrations-before-serve, secret isolation

1. The Docker Stack

Three containers, one command.

Q1. Describe your Docker setup.

EASY

Answer: Three services via Docker Compose: postgres (PostgreSQL 16 with a persistent named volume), backend (FastAPI/uvicorn), and frontend (React built, served by nginx). One command, 'docker compose up --build', runs the whole stack with no host Python, Node, or Postgres required.

Simple explanation: postgres + backend + frontend, started together with one command.

```
Browser -> localhost:3000 -> frontend (nginx)
Browser -> localhost:8000 -> backend (FastAPI)
backend -> postgres:5432 -> PostgreSQL
```

Interview tip: Lead with 'three services, one command, nothing needed on the host'.

Q2. Why three separate containers instead of one?

MEDIUM

Answer: They have different runtimes and lifecycles (Python vs Node/nginx vs a database), should scale and restart independently, and one-container-does-everything is an anti-pattern. Separate images stay small and mirror a realistic deployment topology.

Simple explanation: Different jobs, different runtimes — keep them isolated.

Likely follow-ups:

- Trade-off? (slightly more orchestration config, but far cleaner separation)

Q3. Why is PostgreSQL in a container with a named volume?

EASY

Answer: So nobody needs to install Postgres locally, and the named volume persists data across container restarts and rebuilds. 'docker compose down' keeps the volume; 'down -v' deletes it for a clean slate.

Simple explanation: Containerized DB for easy setup; the volume keeps data safe between restarts.

Project example: The seeded gateway key and logged requests survive a normal down/up because the pgdata volume is retained.

2. Startup & Networking

How the containers find each other and come up safely.

Q4. How does the backend connect to PostgreSQL in Docker?

MEDIUM

Answer: By the Docker service name, not localhost. Compose injects DATABASE_URL as postgresql+psycopg://...@postgres:5432/... so the backend resolves 'postgres' on the Compose network. This overrides any localhost value in the local .env.

Simple explanation: Inside Docker, 'postgres' is the hostname; 'localhost' would be the backend itself.

```
DATABASE_URL=postgresql+psycopg://gateway:***@postgres:5432/gateway
```

Interview tip: Service-name DNS on the Compose network is the key concept here.

Q5. Why is the frontend built with http://localhost:8000 and NOT http://backend:8000?

HARD

Answer: Because the code that calls the backend runs in the user's BROWSER, not inside a container. The browser can't resolve the Docker-internal name 'backend'; it reaches the backend through the published host port localhost:8000. So VITE_API_BASE_URL is a build-time value pointing at the host-reachable URL.

Simple explanation: The browser is outside Docker, so it uses localhost, not the internal service name.

Project example: We verified the built bundle contains http://localhost:8000 and not http://backend:8000.

Likely follow-ups:

- Why is it build-time? (Vite inlines import.meta.env at build, so it must be set before npm run build)

Interview tip: This 'browser is outside the network' insight is a classic Docker gotcha — nail it.

Q6. How do you guarantee the DB is ready and migrated before serving?

MEDIUM

Answer: Postgres has a pg_isready healthcheck; the backend depends_on postgres with condition: service_healthy, so it starts only once the DB is healthy. The backend entrypoint then runs 'alembic upgrade head' before uvicorn, with set -e so a failed migration stops the container instead of serving a broken schema.

Simple explanation: Wait for DB healthy -> migrate -> only then start the server.

```
postgres healthy -> backend starts -> alembic upgrade head -> uvicorn 0.0.0.0:8000
```

Likely follow-ups:

- What binds uvicorn to 0.0.0.0? (so the server is reachable from outside the container, not just inside)

3. Serving, Health & Secrets

Production serving, health checks, and keeping secrets out of images.

Q7. Why serve the frontend with nginx (multi-stage build) instead of 'vite preview'?

MEDIUM

Answer: Stage 1 uses Node to run 'npm ci' and 'npm run build' into static files; stage 2 copies only those files into a small nginx image to serve them. nginx is a real, efficient static server; 'vite preview' is a dev tool, not meant for production. Multi-stage also keeps Node and node_modules out of the final image.

Simple explanation: Build with Node, ship only the static files on nginx — small and production-appropriate.

Project example: nginx also serves a /healthz endpoint used as the container healthcheck, and uses an SPA fallback to index.html.

Likely follow-ups:

- Why did the healthcheck initially fail? (busybox wget resolved localhost to IPv6; fixed by using 127.0.0.1)

Q8. How are secrets kept out of images and the build context?

HARD

Answer: Secrets come from the gitignored backend/.env via Compose env_file at runtime, never baked into images. .dockerignore files exclude .env, .venv, node_modules, dist, caches and .git from the build context. Only the non-secret VITE_API_BASE_URL is passed as a frontend build arg. We scanned the frontend bundle and image to confirm no provider or gateway keys are present.

Simple explanation: Secrets are injected at runtime, excluded from build context, and never in the frontend.

Project example: docker compose ps shows all three services healthy; the backend reads keys from env_file, while the frontend image contains only static files.

Interview tip: Mention .dockerignore explicitly — it's an easy way to accidentally ship .env if forgotten.